

[illegible]

Figure 1: Connecting the SFS module with the pen drive partition

```

    }

    info->used_blocks = used_blocks;
    info->vfs_sb->s_fs_info = info;
    return 0;
}

void shut_browsing(sfs_info_t *info) {
    if (info->used_blocks)
        vfree(info->used_blocks);
}

```

Similarly, all other functions in *browse_real_sfs.c* would also have to be transformed, one by one. Also, note the read from the underlying block device is being captured by the two functions, namely *read_sb_from_real_sfs()* and *read_from_real_sfs()*, which are coded as follows:

```
#include <linux/buffer_head.h> /* struct buffer_head, sb_bread,
... */

#include <linux/string.h> /* For memcpy */

#include "real_sfs_ds.h"

static int read_sb_from_real_sfs(sfs_info_t *info, sfs_super_
block_t *sb) {
    struct buffer_head *bh;

    if (!(bh = sb_bread(info->vfbs_sb, 0 /* Super block is the 0th
block */))) {
        return -EIO;
    }

    memcpy(sb, bh->b_data, SIMULA_FS_BLOCK_SIZE);
    brelse(bh);
    return 0;
}

static int read_from_real_sfs(sfs_info_t *info, byte4_t block,
byte4_t offset, void *buf, byte4_t len) {
    byte4_t block_size = info->sb.block_size;
    struct buffer_head *bh;

    if (offset >= block_size) {
        block += (offset / block_size);
```

```

        offset %= block_size;
    }
    if (offset + len > block_size) // Should never happen {
        return -EINVAL;
    }
    if (!(bh = sb_bread(info->vfs_sb, block))) {
        return -EIO;
    }
    memcpy(buf, bh->b_data + offset, len);
    brelse(bh);
    return 0;
}

```

All the above code pieces put in together as the *real_sfs_minimal.c* (based on the file *real_sfs_bb.c* created earlier), *real_sfs_ops.c*, *real_sfs_ds.h* (based on the same file created earlier), *real_sfs_ops.h*, and a supporting *Makefile*, along with the previously created *format_real_sfs.c* application, are available from http://www.format.com/article_source_code/oct12/file_system_block_drive.zip

Real SFS on block device

Once compiled using *make*, Pugs didn't expect the *real_sfs_first.ko* driver to be way different from the previous *real_sfs_bb.ko* driver, but hoped it would now read and verify the underlying partition. For that, he first tried mounting the usual partition of a pen drive to get an 'Invalid SFS detected' message in *dmesg*; and then tried after formatting it. Note that the same error of 'Not a directory', etc, as in the previous article, still exists, as this driver is still very similar to the previous bare-bones driver. The core functionality's yet to be implemented; it's just that it is now on a real block device partition. Figure 1 shows the exact commands for all these steps.

Note that the `/format_real_sfs` and `mount` commands may take a lot of time (maybe in minutes), depending on the partition size—so preferably use a partition that's less than, say, 1 MB.

With this important step of getting the file system module interacting with the underlying block device, the last step for Pugs would be to do the other transformations from `browse_real_sfs.c` and, accordingly, use them in the SFS module.  

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medalist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at <http://profession.sanka-pugs.com/>, including information about his open source hardware freak-outs, and his various Linux related training sessions and workshops. He can be reached at email@sanka-pugs.com.